

APPENDIX 13 - WEB SCRAPING SCRIPT

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import requests
import logging
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urlparse
import time
from pathlib import Path
from typing import Dict, List, Optional, Set
import re
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager
from selenium.common.exceptions import TimeoutException, WebDriverException

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('scraping.log'),
        logging.StreamHandler()
    ]
)

class RateLimiter:
    def __init__(self, calls_per_minute: int = 30):
        self.calls_per_minute = calls_per_minute
        self.interval = 60 / calls_per_minute
        self.last_call = 0

    def wait(self):
        now = time.time()
        elapsed = now - self.last_call
        if elapsed < self.interval:
            time.sleep(self.interval - elapsed)
        self.last_call = time.time()

class WebScraper:
    def __init__(self, base_url: str, output_dir: str = 'scraped_data'):
        self.base_url = base_url
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(exist_ok=True)

        # Initialize Selenium WebDriver
        chrome_options = Options()
        chrome_options.add_argument('--headless')
        chrome_options.add_argument('--disable-gpu')
        chrome_options.add_argument('--no-sandbox')
        chrome_options.add_argument('--disable-dev-shm-usage')
```

```

try:
    service = Service(ChromeDriverManager().install())
    self.driver = webdriver.Chrome(service=service,
options=chrome_options)
    self.wait = WebDriverWait(self.driver, 10)
except WebDriverException as e:
    logging.error(f"Failed to initialize Selenium WebDriver:
{str(e)}")
    raise

# Initialize requests session
self.session = requests.Session()
self.session.headers.update({
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36',
    'Accept':
'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
    'Accept-Language': 'en-US,en;q=0.5',
})

self.rate_limiter = RateLimiter(calls_per_minute=30)

# Research-relevant keywords for content analysis
self.topic_keywords = {
    'organizational_info': [
        'about us', 'mission', 'vision', 'values', 'goals',
'objectives',
        'history', 'story', 'founded', 'established', 'leadership',
'team'
    ],
    'governance': [
        'governance', 'cooperative', 'commons', 'peer', 'open',
'decision-making', 'board', 'leadership', 'participatory',
'accountability', 'inclusion', 'structure', 'organization',
'democratic', 'community-led', 'ownership', 'stakeholders'
    ],
    'impact': [
        'impact', 'income', 'innovation', 'jobs', 'skills',
'equity',
        'inclusion', 'local', 'outcomes', 'success', 'metrics',
'social',
        'benefits', 'value', 'wealth', 'achievements', 'projects',
'services', 'technology', 'innovation', 'community',
'citizens',
        'customers', 'stakeholders', 'collective', 'rights',
'justice',
        'economic', 'environmental', 'democracy'
    ],
    'resources': [
        'funding', 'support', 'revenue', 'financial',
'sustainability',
        'resources', 'invest', 'income', 'grants', 'donors'
    ],
    'partners': [
        'partners', 'partnerships', 'collaborators',
'collaboration',
        'alliance', 'network', 'ecosystem', 'supporters'
    ],
    'technology': [

```

```

        'data', 'technical documentation', 'code', 'repository',
        'user interface', 'platform', 'open-source', 'technology',
        'infrastructure', 'software', 'digital'
    ],
    'organization_type': [
        'platform cooperative', 'worker-owned platform',
        'member-owned platform', 'cooperative platform', 'data
sovereignty',
        'digital rights', 'data governance', 'data ownership',
        'data trust', 'data commons', 'commons', 'digital commons',
        'peer production', 'open source', 'shared resources',
        'collaborative production', 'participatory governance',
        'democratic governance', 'community governance',
        'collective decision-making'
    ],
    'regulation': [
        'intellectual property', 'government', 'regulation',
        'data protection', 'market'
    ],
    'economic_sector': [
        'product', 'service', 'agriculture', 'farming', 'finance',
        'manufacturing', 'food', 'health', 'education', 'online
store',
        'digital platform', 'retail', 'textile', 'utilities',
        'engineering', 'restaurant', 'tourism', 'banking',
        'insurance',
        'transportation', 'mobility', 'sector', 'research'
    ]
}

```

```

def __del__(self):
    if hasattr(self, 'driver'):
        try:
            self.driver.quit()
        except Exception as e:
            logging.error(f"Error closing Selenium WebDriver:
{str(e)}")

def _should_crawl_link(self, url: str) -> bool:
    try:
        if not url.startswith(self.base_url):
            return False

        # Skip unwanted file types and pages
        skip_patterns = ['.pdf', '.jpg', '.png', '/login', '/cart',
'contact']
        if any(pattern in url.lower() for pattern in skip_patterns):
            return False

        # Prioritize important pages
        priority_patterns = [
            'about', 'governance', 'team', 'leadership', 'impact',
            'projects', 'mission', 'vision', 'values', 'partners',
            'community', 'reports', 'publications'
        ]
        return any(pattern in url.lower() for pattern in
priority_patterns)
    except Exception:

```

```

        return False

    def get_page_content(self, url: str) -> Optional[BeautifulSoup]:
        try:
            # Try Selenium first for JavaScript-rendered content
            self.driver.get(url)
            self.wait.until(EC.presence_of_element_located((By.TAG_NAME,
'body'))))
            page_source = self.driver.page_source
            return BeautifulSoup(page_source, 'lxml')
        except Exception as e:
            logging.warning(f"Selenium fetch failed for {url}, falling back
to requests: {str(e)}")

            # Fallback to requests
            retries = 3
            backoff_factor = 2

            for attempt in range(retries):
                try:
                    self.rate_limiter.wait()
                    response = self.session.get(url)
                    response.encoding = response.apparent_encoding

                    if response.status_code == 429:
                        retry_after = int(response.headers.get('Retry-
After', 30))

                        time.sleep(retry_after)
                        continue

                    response.raise_for_status()
                    return BeautifulSoup(response.text, 'lxml')

                except requests.RequestException as e:
                    wait_time = backoff_factor ** attempt
                    if attempt == retries - 1:
                        logging.error(f"Failed to retrieve {url}:
{str(e)}")

                        return None
                        time.sleep(wait_time)

            return None

    def _extract_content_recursive(self, url: str, visited_urls: Set[str],
depth: int) -> Optional[Dict]:
        if depth > 4 or url in visited_urls:
            return None

        visited_urls.add(url)
        soup = self.get_page_content(url)
        if not soup:
            return None

        try:
            content = {
                'url': url,
                'headings': self._extract_headings(soup),
                'links': self._extract_links(soup),
                'text': self._extract_texts(soup),

```

```

        'metadata': self._extract_metadata(soup),
        'sub_pages': []
    }

    if depth < 4:
        for link in content['links']:
            if self._should_crawl_link(link['url']) and link['url']
not in visited_urls:
                sub_content = self._extract_content_recursive(
                    link['url'], visited_urls, depth + 1
                )
                if sub_content:
                    content['sub_pages'].append(sub_content)

    return content
except Exception as e:
    logging.error(f"Error extracting content from {url}: {str(e)}")
    return None

def extract_content(self) -> Optional[Dict]:
    return self._extract_content_recursive(self.base_url, set(), 0)

def _extract_headings(self, soup: BeautifulSoup) -> List[Dict]:
    headings = []

    # Extract all heading levels
    for tag in soup.find_all(['h1', 'h2', 'h3', 'h4', 'h5', 'h6']):
        headings.append({
            'level': tag.name,
            'text': tag.text.strip()
        })

    # Look for semantic headings
    heading_classes = ['title', 'heading', 'header', 'section-title']
    for cls in heading_classes:
        for element in soup.find_all(class_=re.compile(cls, re.I)):
            if element.name not in ['h1', 'h2', 'h3', 'h4', 'h5',
'h6']:
                headings.append({
                    'level': 'semantic',
                    'text': element.text.strip()
                })

    return headings

def _extract_links(self, soup: BeautifulSoup) -> List[Dict]:
    links = []
    for link in soup.find_all('a', href=True):
        try:
            href = link['href']
            full_url = urljoin(self.base_url, href)
            links.append({
                'text': link.text.strip(),
                'url': full_url
            })
        except Exception as e:
            logging.warning(f"Invalid link {href}: {str(e)}")
    return links

```

```

def _extract_texts(self, soup: BeautifulSoup) -> List[str]:
    text_elements = []

    # Define elements that commonly contain meaningful text
    content_tags = [
        'p', 'article', 'section', 'div', 'span', 'li',
        'td', 'main', 'aside', 'blockquote'
    ]

    content_classes = [
        'content', 'text', 'description', 'body', 'main',
        'article', 'post', 'entry'
    ]

    # Extract text from standard content tags
    for tag in soup.find_all(content_tags):
        if not any(parent.name in content_tags for parent in
tag.parents):
            text = tag.text.strip()
            if text and len(text) > 30: # Minimum length threshold
                text_elements.append(text)

    # Extract text from elements with content-related classes
    for cls in content_classes:
        for element in soup.find_all(class_=re.compile(cls, re.I)):
            if not any(parent.name in content_tags for parent in
element.parents):
                text = element.text.strip()
                if text and len(text) > 30:
                    text_elements.append(text)

    return list(dict.fromkeys(text_elements)) # Remove duplicates

def _extract_metadata(self, soup: BeautifulSoup) -> Dict:
    metadata = {}
    for tag in soup.find_all('meta'):
        name = tag.get('name', tag.get('property', ''))
        content = tag.get('content', '')
        if name and not any(term in name.lower()
            for term in ['social', 'twitter', 'facebook',
'email']):
            metadata[name] = content
    return metadata

def _categorize_links(self, links: List[Dict]) -> Dict:
    categories = {
        'internal': [],
        'external': {
            'social_media': [],
            'blogs': [],
            'news': [],
            'publications': [],
            'partners': [],
            'email': [],
            'other': []
        }
    }

```

```

        social_patterns = ['twitter.com', 'facebook.com', 'linkedin.com',
'instagram.com']
        blog_patterns = ['blog', 'medium.com', 'wordpress']
        news_patterns = ['news', 'press', 'media']
        publication_patterns = ['publication', 'report', 'pdf', 'research']
        partner_patterns = ['partner', 'affiliate', 'collaborator']

        for link in links:
            url = link['url'].lower()
            if url.startswith(self.base_url):
                categories['internal'].append({
                    'text': link['text'],
                    'url': link['url']
                })
            elif url.startswith('mailto:'):
                categories['external']['email'].append(url)
            else:
                link_data = {'text': link['text'], 'url': link['url']}
                if any(pattern in url for pattern in social_patterns):
                    categories['external']['social_media'].append(link_data)
                    elif any(pattern in url for pattern in blog_patterns):
                        categories['external']['blogs'].append(link_data)
                    elif any(pattern in url for pattern in news_patterns):
                        categories['external']['news'].append(link_data)
                    elif any(pattern in url for pattern in
publication_patterns):
                        categories['external']['publications'].append(link_data)
                        elif any(pattern in url for pattern in partner_patterns):
                            categories['external']['partners'].append(link_data)
                        else:
                            categories['external']['other'].append(link_data)

        return categories

    def _analyze_keywords(self, texts: List[str]) -> Dict:
        combined_text = ' '.join(texts).lower()
        results = {}

        for topic, keywords in self.topic_keywords.items():
            found_keywords = []
            for keyword in keywords:
                if keyword.lower() in combined_text:
                    found_keywords.append(keyword)
            if found_keywords:
                results[topic] = found_keywords

        return results

    def save_content(self, content: Dict) -> None:
        """
        Save extracted content with minimal whitespace while maintaining
        readability.
        Uses controlled line breaks between sections and content blocks.
        """
        if not content:
            return

```

```

timestamp = time.strftime('%Y%m%d_%H%M%S')
filename = self.output_dir / f'analysis_{timestamp}.txt'

try:
    with open(filename, 'w', encoding='utf-8') as file:
        def write_page_content(page_content: Dict, level: int = 0):
            indent = "  " * level

            # Page URL
            file.write(f"{indent}Page URL:
{page_content['url']}\n")

            # Headings
            if page_content['headings']:
                file.write(f"{indent}=== HEADINGS ===\n")
                for heading in page_content['headings']:
                    file.write(f"{indent}{heading['level']}:
{heading['text']}\n")

            # Links
            categorized_links =
self._categorize_links(page_content['links'])
            if categorized_links['internal'] or
any(categorized_links['external'].values()):
                file.write(f"{indent}=== LINKS ===\n")

                if categorized_links['internal']:
                    file.write(f"{indent}Internal Links:\n")
                    for link in categorized_links['internal']:
                        file.write(f"{indent}- {link['text']}:
{link['url']}\n")

                has_external =
any(categorized_links['external'].values())
                if has_external:
                    file.write(f"{indent}External Links:\n")
                    for category, links in
categorized_links['external'].items():
                        if links:

file.write(f"{indent}{category.replace('_', ' ').title()}: \n")
                        for link in links:
                            if isinstance(link, dict):
                                file.write(f"{indent}-
{link['text']}: {link['url']}\n")
                            else:
                                file.write(f"{indent}-
{link}\n")

            # Text Content
            if page_content['text']:
                file.write(f"{indent}=== TEXT CONTENT ===\n")
                for text in page_content['text']:
                    file.write(f"{indent}{text}\n")

            # Metadata
            if page_content['metadata']:
                file.write(f"{indent}=== METADATA ===\n")
                for key, value in page_content['metadata'].items():

```



```

        file.write(f"{indent}{key}: {value}\n")

    # Sub-pages
    if page_content['sub_pages']:
        file.write(f"{indent}=== SUB-PAGES ===\n")
        for sub_page in page_content['sub_pages']:
            write_page_content(sub_page, level + 1)

    # Write main content
    write_page_content(content)

    # Keyword Analysis
    file.write("=== KEYWORD ANALYSIS ===\n")
    all_texts = content['text']
    for sub_page in content.get('sub_pages', []):
        all_texts.extend(sub_page['text'])

    keyword_analysis = self._analyze_keywords(all_texts)
    for topic, found_keywords in keyword_analysis.items():
        if found_keywords:
            file.write(f"{topic.replace('_', ' ').title()}\n")
            for keyword in found_keywords:
                file.write(f"- {keyword}\n")

    logging.info(f"Analysis saved to {filename}")
except IOError as e:
    logging.error(f"Failed to save content: {str(e)}")

def main():
    """Main execution function with error handling."""
    try:
        os.chdir('/Users/dhruvdeepak/Desktop/Python Directory')
    except OSError as e:
        logging.error(f"Failed to change directory: {str(e)}")
        return

    url = "https://www.loomio.com" # Example URL

    try:
        scraper = WebScraper(url)
        content = scraper.extract_content()
        if content:
            scraper.save_content(content)
            logging.info("Content extraction and analysis completed successfully")
        else:
            logging.error("Failed to extract content")
    except Exception as e:
        logging.error(f"An error occurred: {str(e)}")
        import traceback
        logging.error(traceback.format_exc())

if __name__ == "__main__":
    main()

```